Agente investigador en las revistas de Optica Publishing Gruop

Resumen

Adaptamos un agente investigador multiagente para que, además de buscar en la web, arXiv y Wikipedia, consulte artículos open access de las revistas de Optica Publishing Group (Optics Express, Optica, Biomedical Optics Express, entre otras). El agente corre en local con Docker y un modelo de lenguaje servido por Ollama, sin depender de APIs de pago para el modelo.

Los cambios se agrupan en tres frentes: tres herramientas nuevas de búsqueda académica, instrucciones explícitas para que el agente use Optica en temas de óptica y fotónica, y ajustes de estabilidad para que el flujo completo termine con un modelo local de 4B parámetros.

Nuevas funciones

Creamos el módulo `src/optica_tools.py` con tres herramientas que el research agent puede llamar por sí mismo. Ninguna descarga contenido de pago: solo enlaza versiones open access legales.

Herramienta	Qué hace	Fuente de datos	Requiere clave
`optica_open_access_search`	Busca artículos open access publicados por Optica Publishing Group y devuelve título, autores, revista, año, DOI, enlace al PDF, licencia, número de citas y resumen. Permite filtrar desde un año.	API de OpenAlex	No (solo un correo de contacto)
`site_search`	Búsqueda web restringida a sitios de interés, por defecto opg.optica.org, arxiv.org y spie.org.	Tavily	Sí (Tavily)
`check_open_access`	Dado un DOI, indica si existe una versión gratuita y legal y dónde está el PDF.	API de Unpaywall	No (solo un correo de contacto)

Las tres devuelven los errores como datos en lugar de lanzar excepciones, para que un fallo de red no detenga el flujo del agente. Los sitios de `site_search` se configuran con la variable `RESEARCH_SITES` del archivo `.env`.

Órdenes para buscar en Optica

Agregar las herramientas no basta: el modelo tiene que saber cuándo usarlas. Por eso dimos instrucciones explícitas en dos niveles.

En el planner (`src/planning_agent.py`). Le informamos que el research agent puede buscar artículos open access de Optica y verificar DOIs. Además, le ordenamos que, si el tema es de óptica o fotónica (láseres, imagen, espectroscopía, metamateriales, fibra óptica, etc.), el tercer paso del plan sea obligatoriamente:

> *Research agent: Search open access articles from Optica Publishing Group journals on the topic and record title, authors, journal, year, DOI and free PDF link.*

En el research agent (`src/agents.py`). Añadimos las tres herramientas a su lista y las describimos en su prompt, junto a Tavily, arXiv y Wikipedia. Las reglas de selección incluyen dos órdenes nuevas:

- Para temas de óptica o fotónica, usar siempre `optica_open_access_search` y preferir fuentes open access.
- Para artículos con DOI pero sin enlace gratuito, usar `check_open_access`.

Así, una consulta como "photonic crystals" genera un plan con un paso dedicado a Optica, y en ese paso el agente consulta las revistas de Optica y registra los enlaces a los PDF gratuitos.

Ajustes para el modelo local

Con un modelo local, el flujo fallaba por tiempo de espera en el paso del writer. Estos son los problemas que encontramos y cómo los resolvimos.

Problema	Causa	Solución
El contenedor no arrancaba (`No module named 'psycopg'`)	SQLAlchemy 2.1 usa por defecto el driver `psycopg` 3	Agregar `psycopg[binary]` a `requirements.txt`
Contexto insuficiente	El modelo base tiene 4096 tokens y el flujo acumula investigación, borradores e historial	Modelo propio `qwen3-4b-instruct-16k` con `num_ctx 16384`, creado con un Modelfile
Error `invalid model name`	Comillas, comentarios o el prefijo `ollama:` en `LLM_MODEL`; `--env-file` no los elimina	Líneas del `.env` sin comillas ni comentarios y prefijo `openai:`
Timeout en el writer	El writer esperaba la respuesta completa (hasta 15 000 tokens) en una sola petición	Respuesta en streaming, límite de 4096 tokens y timeout de 900 s
Prompts cada vez más largos	Cada paso recibía todo lo producido antes, incluidos extractos de PDFs	Recorte a 6000 caracteres por paso anterior
Plan genérico, sin el paso de Optica	Los modelos Qwen3 con razonamiento emiten bloques `<think>` que rompen la lectura del plan	Instrucción `/no_think` y eliminación de esos bloques
Esperas triplicadas	El cliente reintentaba en silencio tras cada timeout	Reintentos desactivados (`max_retries=0`)

El prefijo `openai:` no envía nada a OpenAI: indica que el código usa el protocolo compatible con OpenAI que ofrece Ollama en `/v1`, porque el proveedor `ollama:` de aisuite envía parámetros que Ollama rechaza.

Cómo ejecutarlo

Probado en un MacBook Pro M3 con 16 GB de RAM, con Ollama como aplicación nativa (usa la GPU) y Docker Desktop.

1. Crear el modelo con 16k de contexto:

```
ollama pull qwen3:4b-instruct
printf 'FROM qwen3:4b-instruct\nPARAMETER num_ctx 16384\n' > Modelfile
ollama create qwen3-4b-instruct-16k -f Modelfile
```

2. Configurar el archivo `.env` (sin comillas ni comentarios al final de las líneas):

```
LLM_MODEL=openai:qwen3-4b-instruct-16k
OLLAMA_API_URL=http://host.docker.internal:11434
OLLAMA_TIMEOUT=900
NO_THINK=0
TAVILY_API_KEY=<clave de Tavily>
OPENALEX_EMAIL=<correo>
UNPAYWALL_EMAIL=<correo>
RESEARCH_SITES=opg.optica.org,arxiv.org,spie.org
```

3. Construir y correr el contenedor (Postgres corre dentro del mismo contenedor):

```
docker build -t agentic-ai-v2 .
docker run --rm -it -p 8000:8000 --env-file .env --name fpsvc-v2 agentic-ai-v2
```

4. Abrir `http://localhost:8000` y escribir el tema de investigación.

Para verificar la conexión con Ollama: `docker exec -it fpsvc-v2 python scripts/test_ollama_connection.py`.

Prueba y limitaciones

Probamos el agente con el tema "photonic crystals" y el flujo completo terminó sin errores: planificación, investigación, borrador, revisión y reporte final en Markdown con referencias.

Limitaciones observadas:

- **Tiempo:** el flujo tarda varios minutos. La parte más lenta es arXiv, que descarga y extrae el texto de cada PDF. Se puede acelerar trayendo solo los resúmenes (`_INCLUDE_PDF = False` y `_EXTRACT_TEXT = False` en `research_tools.py`).
- **Tamaño del modelo:** un modelo de 4B parámetros a veces no llama a todas las herramientas disponibles o comete errores en las citas. Un modelo mayor, como `qwen3:14b`, mejora la calidad a costa de velocidad.
- **Historial no persistente:** `main.py` borra la tabla de tareas cada vez que arranca el contenedor, así que los reportes deben guardarse antes de detenerlo.
- **Cobertura:** la búsqueda de Optica depende de cómo OpenAlex indexa a la editorial; artículos muy recientes pueden tardar en aparecer.